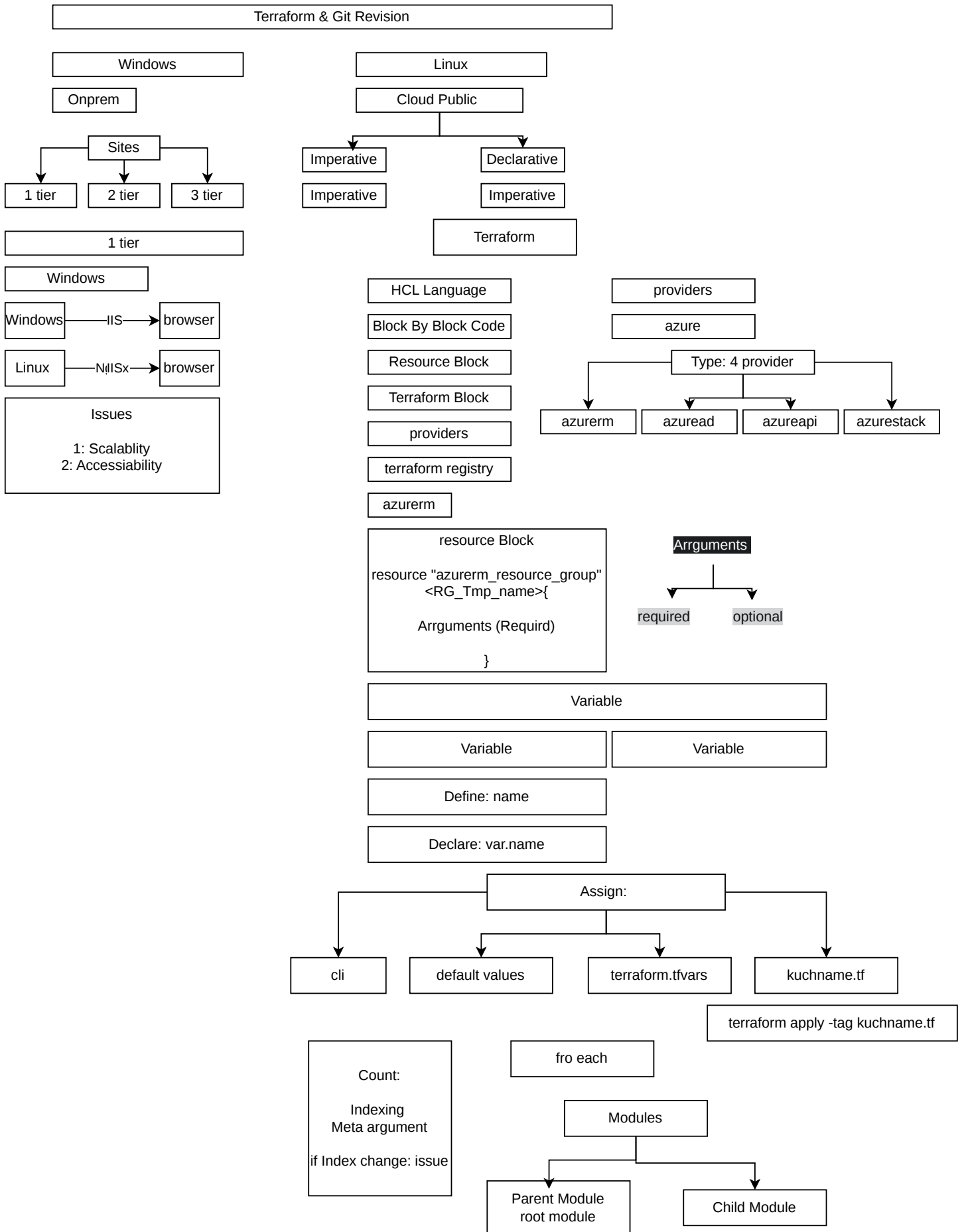
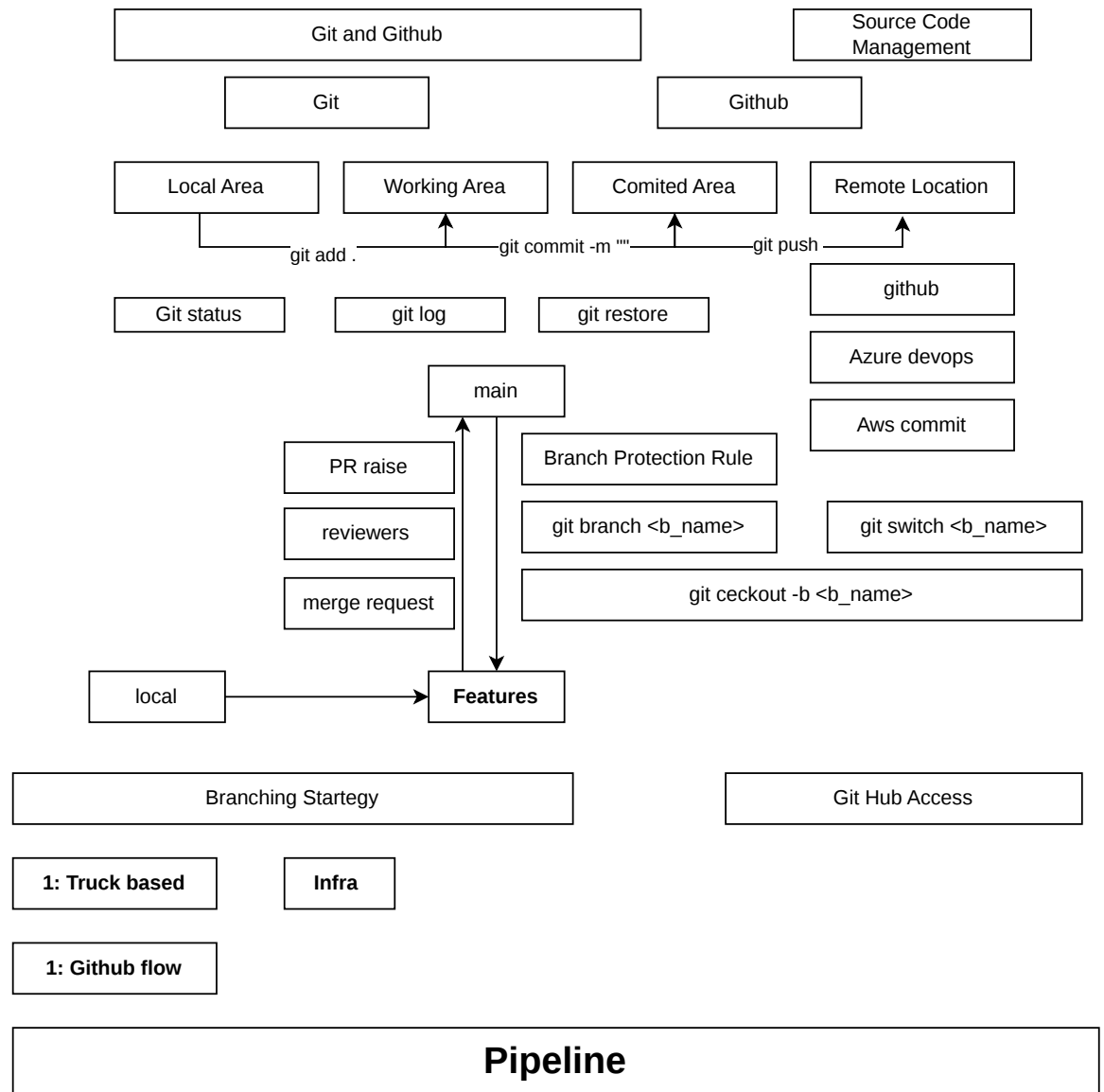






Revision Day 4 (Pipeline)

Pipeline:

- 1: case sensitive
- 2: indentation:
tab = 4 spaces
required: 2 spaces
- 3: array or list: -
- 4: YAML
- 5: key: value

.github/workflows/

1: 1 yaml file honi chaiye?
hm wha multiple yaml file bna skte hai.

why we need to create multiple yaml file?

on: keyword

name:
name keyword

on:
on keyword

Push
PR Raise
Tag
Issues (resolve)

workflow_dispatch
manual trigger

on:
workflow_dispatch:

Automated Time Triggers (Schedule)

on:
schedule:
- cron: '0 0 * * *'

on:
push:
branches: [main, feature]

on:
push:
branches: [main, feature/rg1]

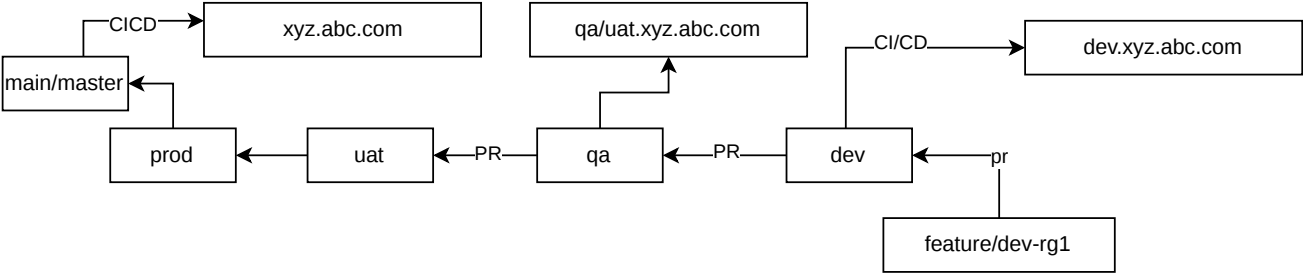
on:
push:
branches: [main, feature/rg2]

feature/rg1
feature/rg2
feature/rg3
feature/vnt1

feature/*

\$default-branch

on:
push:
branches: [main, feature/*]



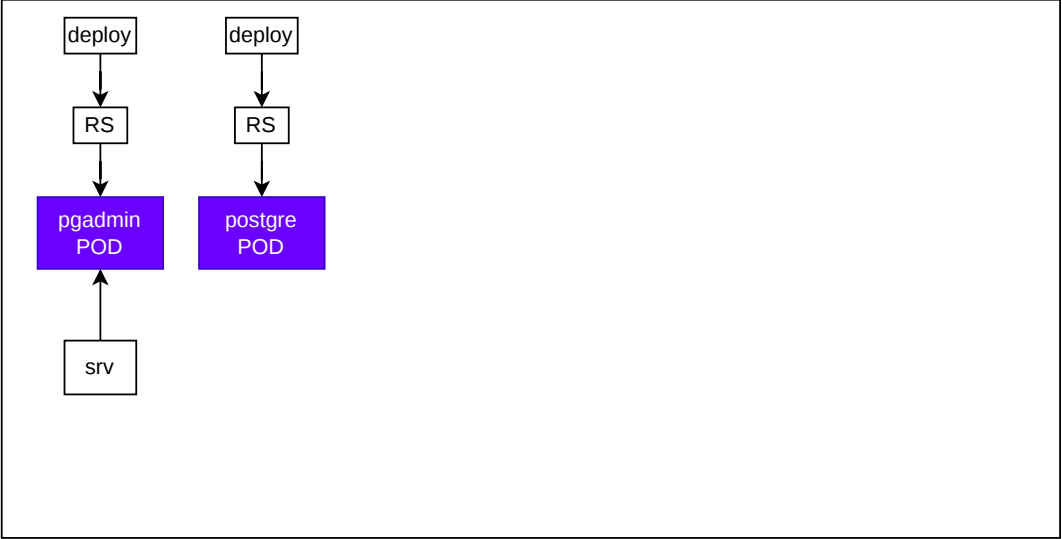
Jobs:
job name:
steps:
step:

runs-on:

Welcome main.tf 3 ! learning.yml M

Leaning > .github > workflows > ! learning.yml

```
1  name: learning
2  on: push
3  jobs:
4    first_job:
5      runs-on: ubuntu-latest
6      steps:
7        - name: want-to-print-something
8          run: echo "Hello World!"
9    second_job:
10     runs-on: windows-latest
11     needs: [first_job, second_job, third_job]
12     steps:
13       - name: want-to-print-something
14         run: echo "Second Hello World!"
15
16
```



Docker

Problem in Docker:

- docker swam: orcestration issue
- aphermal
- 2: rolling update missing
- 3: Small Application

Docker file:

light weight
run install pacages
remove

20 MB

20 + 20 MB

20 + 20 + 20 = 60 MB

final image:
60MB

20 + 20 = 40 MB

Docker file:

light weight
run install pacages && remove

20 MB

20 + 20 MB

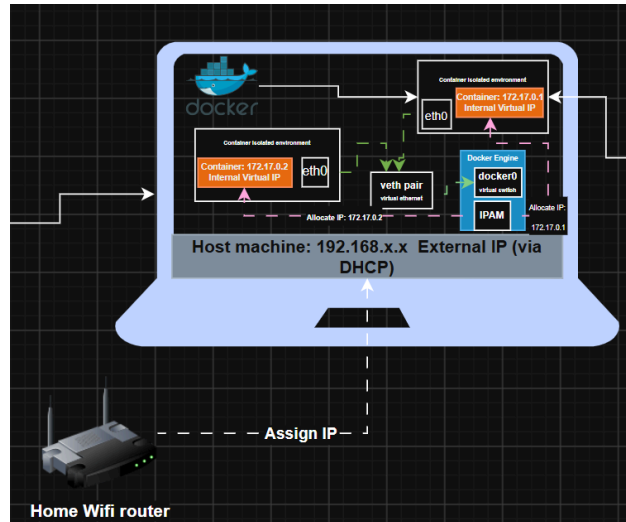
final image:
40MB

20 + 20 = 40MB

Container IP

Container IP

IPAM



K8's

Control Plane

Woker node

API Server

etcd

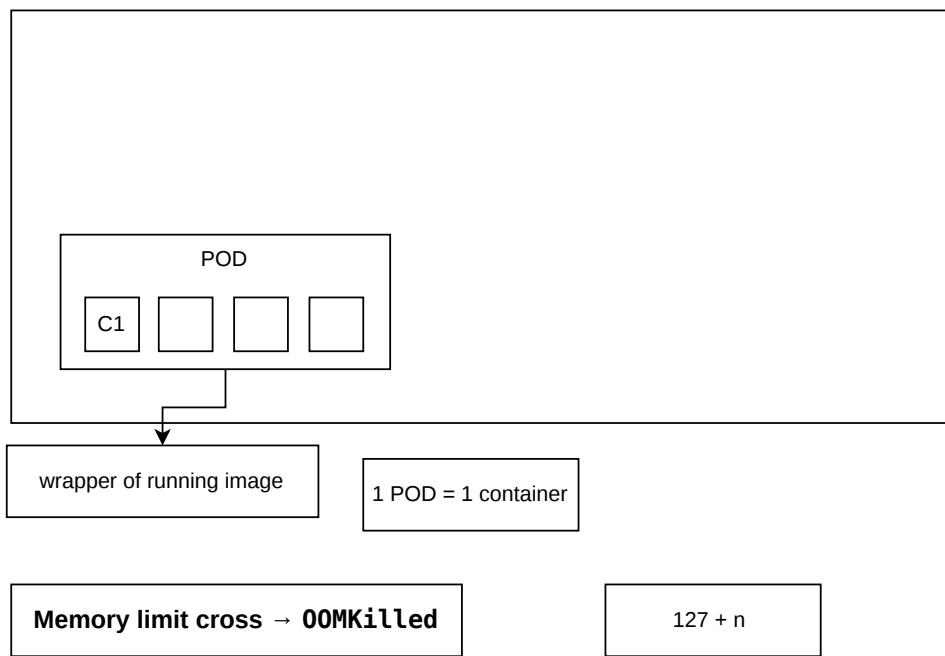
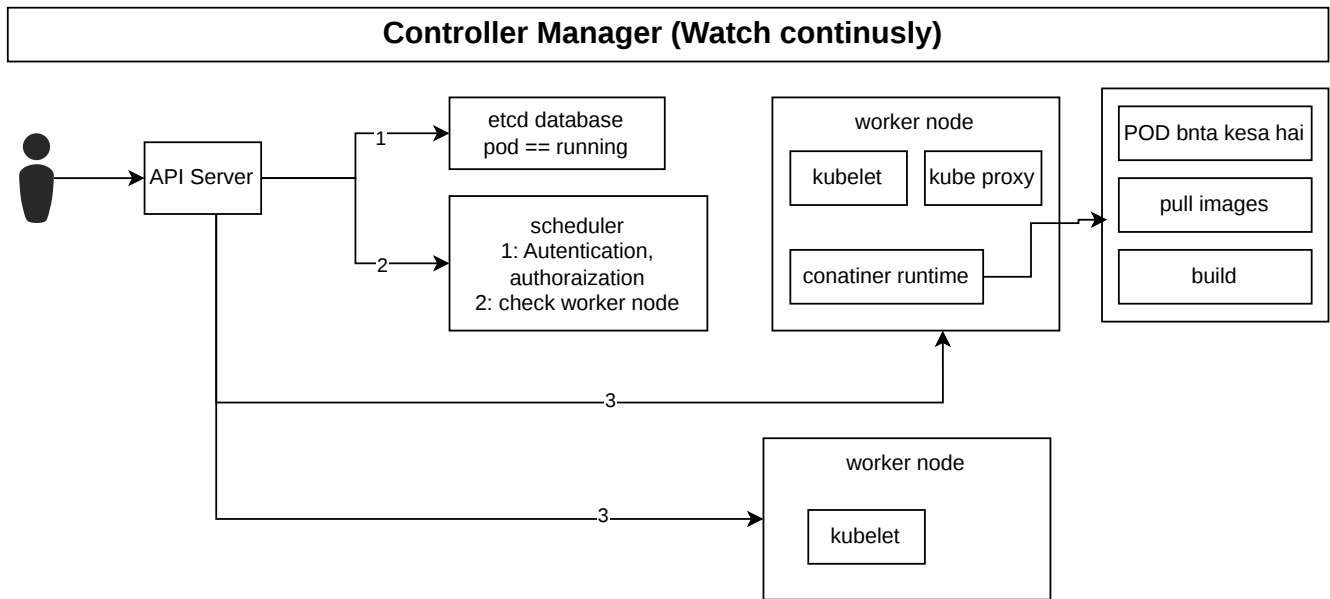
kubelet

kubeproxy

scheduler

controller

container
runtime

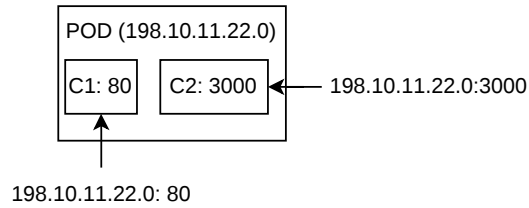


Problem	Result
Memory limit cross	OOMKilled
CPU limit reach	CPU throttling / slow app
CPU request ke liye space nahi	Pending
Memory request ke liye space nahi	Pending
Node memory critically low	MemoryPressure / Eviction
Container repeatedly crash	CrashLoopBackOff
Node resources exhausted	Pod Evicted
Sidecar dependency fail	Application functionality affected

Docker -> Conatiner
(IP assign Private IP)

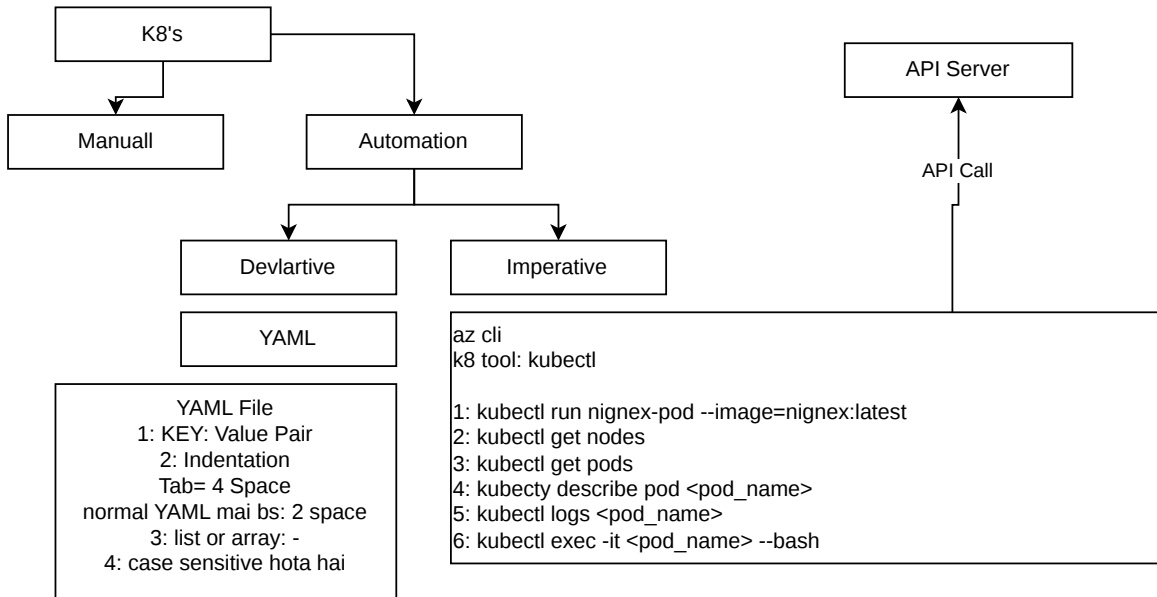
POD = IP (Private ip) -> Conatiner (

Internet → Load Balancer Public IP → Service → Pod.



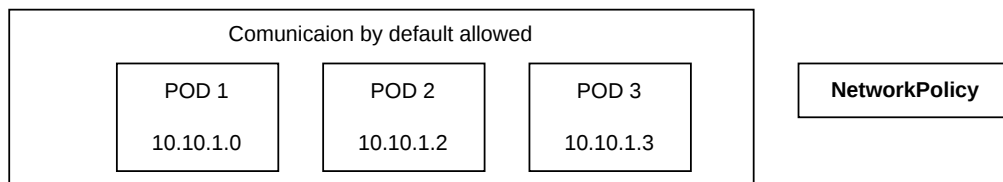
Same Pod = Same Network/IP → same port par 2 containers listen nahi kar sakte.

Different Pods = Different IP → same port use kar sakte hain.



AKMS
A: apiVersion
K: KIND
M: Metadata
S: Spec

AKMS
A: apiVersion
K: KIND
M: Metadata
S: Spec



Node = ek machine/VM

Node Pool = similar Nodes ka group

Cluster = multiple Node Pools + Kubernetes control plane

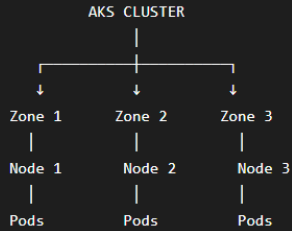
Fleet Manager = multiple AKS clusters ko ek saath manage karne ka central management layer.

Availability zones

Availability Set

3 Availability Zones $\neq$ 3 AKS Clusters

3 Availability Zones = 1 AKS Cluster ke resources/nodes ko 3 zones mein spread karna.



Example

Agar tumne 3 Availability Zones select kiye aur Node Pool mein 3 nodes hain:

```
1 AKS Cluster
├── Zone 1 → Node 1
├── Zone 2 → Node 2
└── Zone 3 → Node 3
```

Agar 6 nodes hain, toh nodes zones mein distribute ho sakte hain:

```
Zone 1 → Node 1, Node 2
Zone 2 → Node 3, Node 4
Zone 3 → Node 5, Node 6
```

AKS Cluster

Node Pool 1

```
├── Node 1 → Zone 1
├── Node 2 → Zone 2
└── Node 3 → Zone 3
```

Node Pool 2

```
├── Node 4 → Zone 1
├── Node 5 → Zone 2
└── Node 6 → Zone 3
```

docker

K8s

IPAM

CNI (Container Network Interface) plugin

Azure CNI (Pod Subnet / traditional Azure CNI)

Node → Azure VNet/Subnet ka IP
Pod → Azure VNet/Subnet se IP

Azure CNI Overlay

Node → Azure VNet/Subnet ka IP
Pod → Separate Pod CIDR ka IP



Calico kya hai?

Calico ek open-source networking + network security solution hai

Cilium

Cilium = Kubernetes networking + security, jo eBPF technology use karta hai.

Flannel

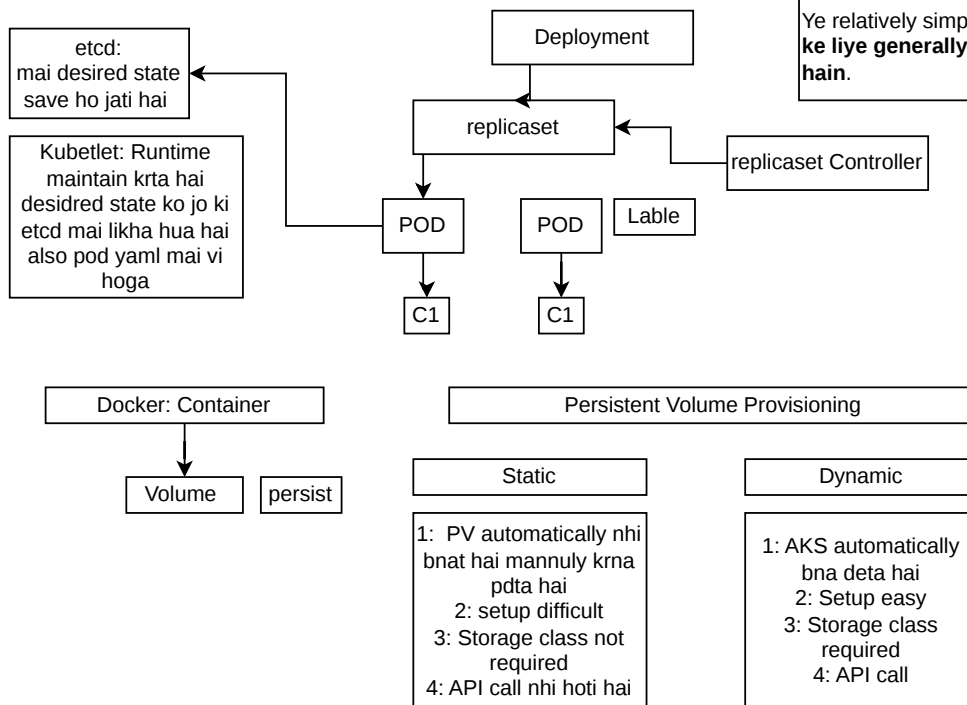
Flannel = simple Kubernetes networking solution.

Iska main focus Pods ko network connectivity dena hai.

```

Pod A
|
Flannel
|
Pod B
  
```

Ye relatively simple hai, lekin advanced NetworkPolicy/security ke liye generally Calico/Cilium jaise solutions use kiye jaate hain.



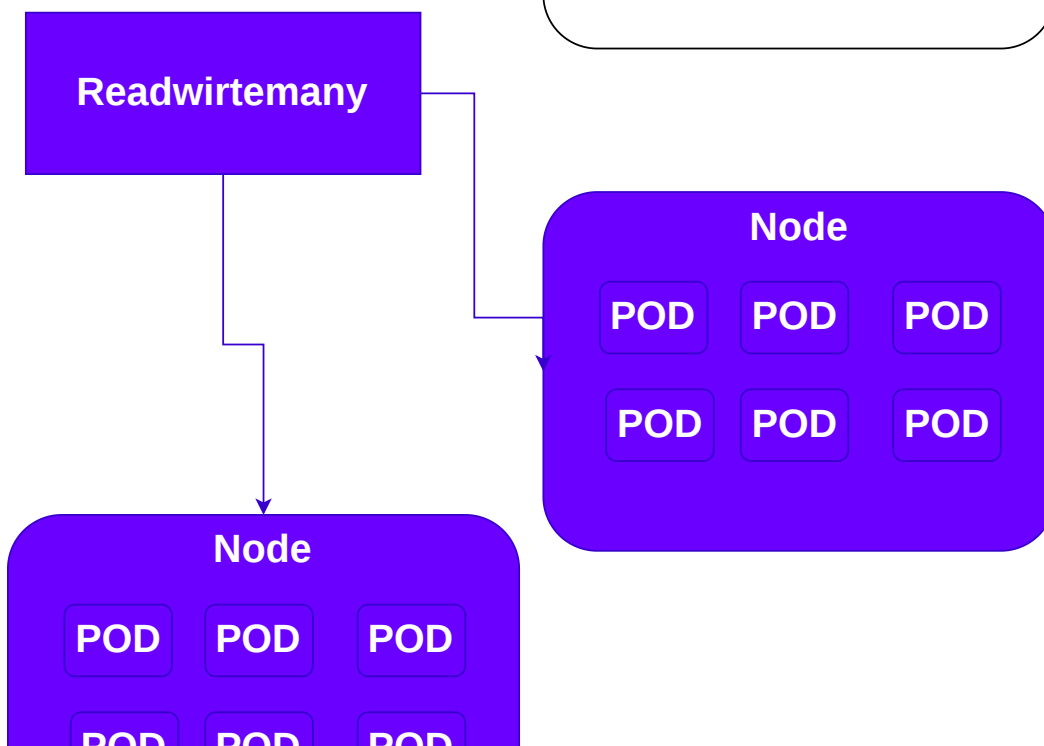
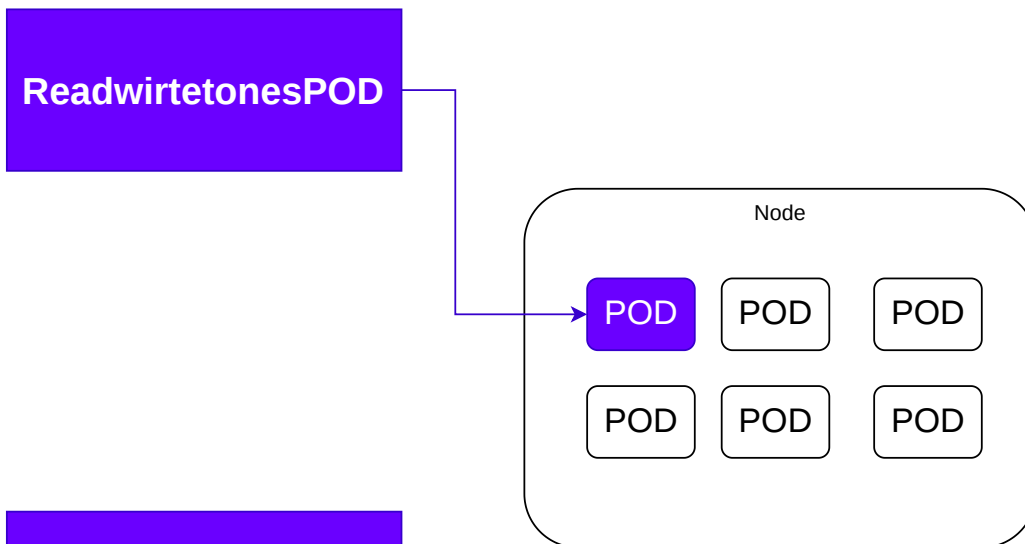
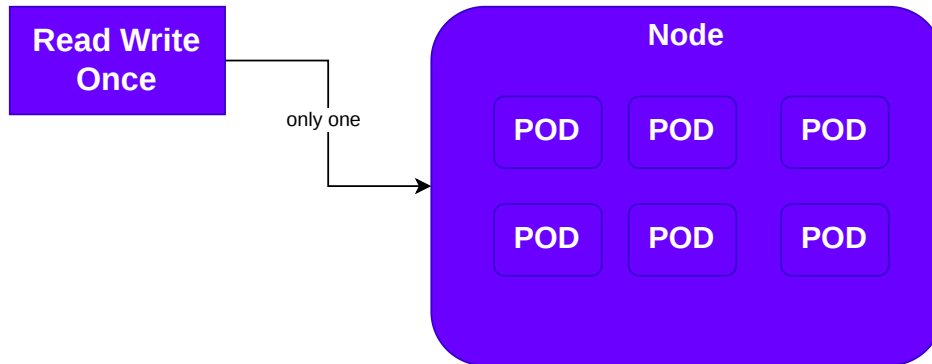
Access Mode	Abbreviation	Description	Common Use Case
ReadWriteOnce	RWO	Mounted as read-write by a single node . Multiple pods on that <i>same</i> node can access it.	Block storage, single-instance databases (e.g., MySQL, PostgreSQL).
ReadWriteMany	RWX	Mounted as read-write by multiple nodes simultaneously.	Shared file systems, content management systems, multi-replica apps needing shared writes.
ReadOnlyMany	ROX	Mounted as read-only by multiple nodes simultaneously.	Shared configuration files, static web assets.



ReadWriteOncePod RWOP

Mounted as read-write by a **single pod** across the whole cluster.

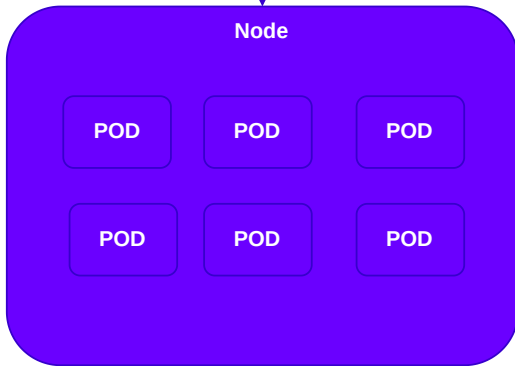
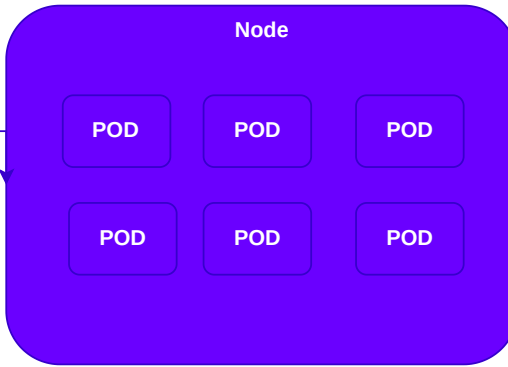
Strict single-writer workloads requiring absolute isolation.





only read

only read



KUBERNETES VOLUME – ACCESS MODES (Explained with Real-World Examples)

Access Modes define how a PersistentVolume can be mounted by pods in the cluster (read/write permission & by how many pods/nodes).

Pod
 Node
 Volume (PV)

1. ReadWriteOnce (RWO)

Volume can be mounted as read-write by a single node (multiple pods on that same node can use it).

✓ Multiple pods on same node – Read/Write OK
✗ Pod on another node – Not Allowed

Who can mount?	• Single Node • Multiple Pods on same node
Mount Type	Read-Write
Use Cases	• Databases (MySQL, PostgreSQL) • Caching (Redis) • Stateful App (single writer)
Real-World Example	A MySQL database pod and a backup sidecar pod on the same node sharing the same volume.

2. ReadWriteOncePod (RWOP)

Volume can be mounted as read-write by ONLY ONE POD in the entire cluster.

✓ Only 1 pod in whole cluster – Read/Write OK
✗ Any other pod (even on same node) – Not Allowed

Who can mount?	• Only One Pod in whole cluster
Mount Type	Read-Write
Use Cases	• Applications that need strict single writer • Legacy apps that don't support multi-attach
Real-World Example	A licensing application that must run in only one pod at a time anywhere in the cluster.

3. ReadOnlyMany (ROX)

Volume can be mounted as read-only by many pods on many nodes.

✓ Many pods on many nodes – Read Only OK
✗ Write Operation – Not Allowed

Who can mount?	• Many Nodes • Many Pods
Mount Type	Read-Only
Use Cases	• Config data • Static content (HTML, JS, assets) • Shared reference data
Real-World Example	Multiple web server pods across nodes serving static website files from the same volume (read-only).

4. ReadWriteMany (RWX)

Volume can be mounted as read-write by many nodes and multiple pods.

✓ Many pods on many nodes – Read/Write OK

Who can mount?	• Many Nodes • Many Pods
Mount Type	Read-Write
Use Cases	• Shared storage for applications • Content management systems • Collaborative workloads
Real-World Example	Multiple pods of a CMS (like WordPress) across nodes reading/writing files (uploaded media, content).

Quick Comparison

Access Mode	Single Node (Multiple Pods)	Multiple Nodes (Multiple Pods)	Read	Write	Typical Backends
ReadWriteOnce (RWO)	✓ Yes	✗ No	✓ Yes	✓ Yes	EBS, PD, Azure Disk, Local PV
ReadWriteOncePod (RWOP)	✗ No (only 1 pod)	✗ No (only 1 pod)	✓ Yes	✓ Yes	EBS (v1 v1.27), PD (v1 v1.27)
ReadOnlyMany (ROX)	✓ Yes	✓ Yes	✓ Yes	✗ No	NFS (not), CephFS (no), Azure Files (no)
ReadWriteMany (RWX)	✓ Yes	✓ Yes	✓ Yes	✓ Yes	NFS, CephFS, Azure Files, GlusterFS, Portworx RWX

How Access Mode is Set?

In PersistentVolume (PV) or StorageClass

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: my-pv
spec:
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: fast-ssd
  owner: abc.cnt.wms.cne
  volumeHandle: vol-8abc123 ...
```

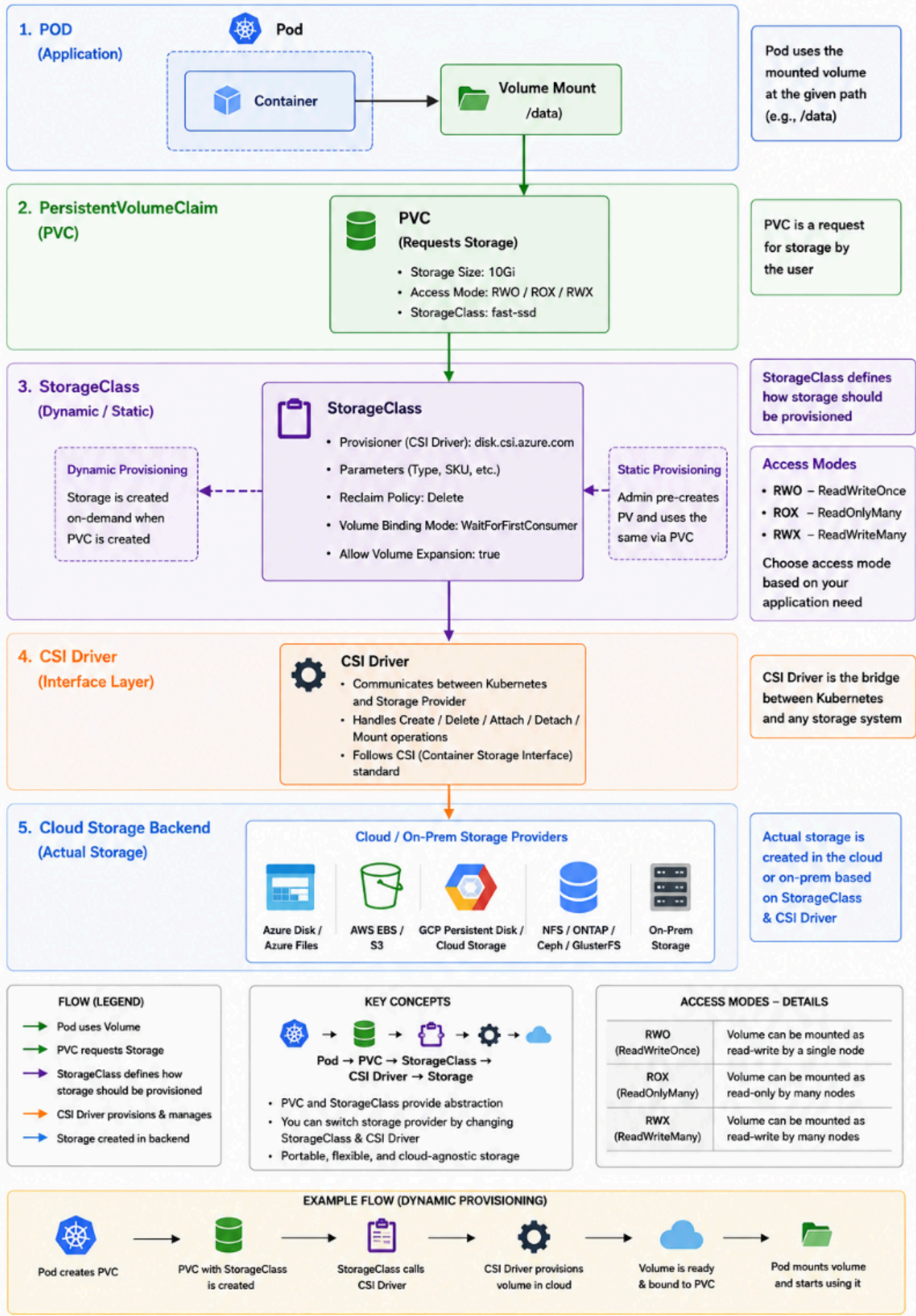
Important Notes

- AccessModes are requested in PVC. The PV must support it.
- Not all storage supports all access modes.
- RWO = single node, multiple pods allowed.
- RWOP = only one pod in entire cluster (stricter than RWO).
- ROX = many can read, but no one can write.
- RWX = many can read and write (needs shared storage backend).

Common Backends & Supported Access Modes

AWS EBS	GCP PD	Azure Disk	NFS	Azure Files	CephFS	GlusterFS
RWO, RWOP	RWO, RWOP	RWO	ROX, RWX	ROX, RWX	ROX, RWX	RWX

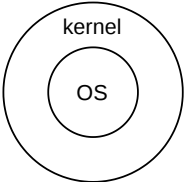
KUBERNETES STORAGE ARCHITECTURE – VOLUME, PVC, STORAGECLASS, CSI DRIVER & CLOUD STORAGE





Linux

OS?
Windows?
Linux?



1 tier applicationo

2 tier applicationo

3 tier applicationo

Linux?
whoamic
mkdir
ls
touch
pwd
cat

Windows:

IIS

C:\inetpub\wwwroot

mv sorce_path destination_path

Copy - cp
Move - mv

cp leaves the original file exactly where it is and creates a brand-new duplicate,

while mv relocates the original file to a new place (or renames it), leaving nothing behind in the old location.

Linux

Killercode

apt update
apt insatl nginx
systemctl status nginx

copy code: git clone URL

sudo :

**Nginx -> path -> /usr/share/nginx/html/ (Docker Conatiner k
ander nginx - /usr/share/nginx/html/**

LINUX: /var/www/html/

Nginx path: /usr/share/nginx/html/

mv sorce_path destination_path

terraform

Dockerfile

Virtualization

Vs

Docker (Containerization)

1: Heavy Weight
2: OS
3: Uptime High

2: Host Machine OS utilisation
3: Light Weight
4: Boottime Fast

Images: Pre configured OS
Blue Print

Docker Image: is also

`docker run ubuntu: latest`

OS - kernel, (file, others)

`docker ps`

`docker images`

`docker stop container_id`

`docker rm con_id`

`docker rmi image_id`

pull nginx image

Docker registry

docker
application
/usr/share/nginx/html/
Port mapping

Custom Image

`docker cp ./index.html hello_word:/usr/share/nginx/html/`

`docker cp source_code_location hello_word:destination_location`

Imperative Way

1: nginx image leke aaye
2: image runnig -> conatiner
3: local code _ artifacts push conatiner k ander
4: commit - photo click kr di
5: nginx delete kr diya

Declarative
Dockerfile

Dockerfile - File
Text file

FROM
COPY
RUN

`docker build -t filename .`

Dockerfile:

FROM image:v1 (base image)

```

1336 docker run -d --name=hello_word nginx:latest
1337 docker ps
1338 docker cp ./index.html hello_word:/usr/share/nginx/html/
1339 docker exec hello_word --help
1340 docker exec --help
1341 docker exec hello_word apt-get update
1342 docker exec echo("hello")
1343 docker exec echo "hello"
1344 docker exec hello_word echo "hello"
1345 docker exec -it hello_word bash
1346 docker commit hello_word overbhaiteam:v1
1347 docker ps
1348 docker images
1349 docker ps
1350 docker stop fdf0fb99fb8f && docker rm fdf0fb99fb8f
1351 docker ps
1352 docker images
1353 docker run -d --name overbhaiteam1 -p 8080:80 overbhaiteam
1354 docker run -d --name overbhaiteam1 -p 8080:80 overbhaiteam:v1
1355 docker ps
1356 clear
1357 docker build --help
1358 clear
1359 docker build -t ovesbharteam:v2 .
1360 docker ps
1361 docker images
1362 docker ps
1363 docker run -d --name overbhaiteam2 -p 8081:80 overbhaiteam:v2
1364 docker run -d --name overbhaiteam2 -p 8081:80 ovesbharteam:v2
1365 docker ps
1366 history

```

Dockerfile

FROM
COPY
WORKDIR: mkdir then cd

The screenshot shows a code editor with two files open: `index.html` and `dockerfile`. The `dockerfile` file contains the following content:

```

1
2 # STEP1: docker run -d --name=hello_word nginx:latest
3 FROM nginx:alpine
4
5 # STEP2: docker cp ./index.html hello_word:/usr/share/nginx/html/
6 COPY ./index.html /usr/share/nginx/html/
7
8
9 # STEP 3 and 4: Commit and delete temp container
10 # docker build -t ovesbharteam:v2 .

```

Powerscript

variables
 print
 child process
 process